



写给你的 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 使用指南

魏舟 著

2026 年 3 月 3 日

谨以此书献给小顾同学。

同时作为学习 $\text{\LaTeX}$ 的一个手册，方便日后查找。

目录

第 1 章	L^AT_EX 简介	1
1.1	为什么选择 L^AT_EX	1
1.2	数学公式示例	1
1.3	L^AT_EX 的安装和配置	2
1.3.1	安装网址和步骤	2
1.3.2	配置 VSCode	3
第 2 章	你好，小顾！	7
2.1	L^AT_EX 文档的基本结构 (Preamble vs. Body)	7
2.1.1	文档类声明: <code>documentclass</code>	7
2.2	宏包 (Package) 的使用与管理	8
2.2.1	加载宏包	8
2.2.2	全局设置	8
2.3	第一个 L^AT_EX 文档示例	9
第 3 章	基础排版语法	11
3.1	标题与分级结构	11
3.1.1	基本标题命令	11
3.1.2	无编号标题	12
3.1.3	标题格式定制	12
3.2	正文输入：换行、分段与特殊字符	13
3.2.1	空白字符与缩进控制	13
3.2.2	换行、换页与间距控制	13
3.2.3	常见的页面布局 (A4)	13
3.2.4	段落与行间距控制	14
3.2.5	常用的长度单位	14
3.2.6	特殊字符与转义	15
3.2.7	注释技巧	15
3.3	字体控制：字号、粗体、斜体与下划线	16
3.3.1	字号控制	16
3.3.2	字体样式	17
3.4	列表环境：有序与无序列表	18
3.4.1	无序列表 (<code>itemize</code>)	18
3.4.2	有序列表 (<code>enumerate</code>)	19

3.4.3 嵌套列表	19
3.5 插入超链接与目录	20
3.5.1 自动生成目录	20
3.5.2 插入超链接	21
3.6 插入脚注	23
3.6.1 插入脚注	24
第 4 章 数学公式排版	25
4.1 行内公式与块级公式	25
4.1.1 行内公式	25
4.1.2 块级公式	25
4.2 数学符号与运算符	26
4.2.1 上下标	26
4.2.2 分数与根号	26
4.3 矩阵、多行方程组与编号控制	27
4.4 妙招	28
4.5 符号大全	28
第 5 章 表格与图片管理	33
5.1 基础表格 (tabular) 与三线表 (booktabs)	33
5.2 图片插入与路径管理	34
5.3 浮动体 (Float) 原理	34
第 6 章 自动化与引用	35
6.1 交叉引用: Label 与 Ref 的自动编号	35
6.2 参考文献: BibTeX 数据库与引用样式	35
6.3 自动生成目录与索引	36

L^AT_EX 简介

Every time I read a LaTeX document, I think, wow, this must be correct!

—Christos Papadimitriou

L^AT_EX 是一种基于 T_EX 的排版系统，由美国计算机学家莱斯利·兰伯特（Leslie Lamport）在 20 世纪 80 年代初期开发。

它并不是一个简单的字处理软件（如 Microsoft Word），而是一种标记语言。用户通过编写代码来描述文档的结构，由编译器（如 pdfLaTeX 或 XeLaTeX）将其转化为精美的 PDF 文档。

缺点很显然：这个东西像编程语言一样，不够直观。但是 Word 等软件的排版效果和公式编辑器的功能远远不如 L^AT_EX，尤其是在处理复杂的数学公式和长文档时。于是人们发明了 Markdown 等更为简洁的标记语言，但它们在专业排版方面仍然无法与 L^AT_EX 相媲美。好吧其实如果这个标题不是 Latex 的话，我认为 Markdown 这种轻排版用起来会更方便些。好吧咱们继续往下说。

§ 1.1

为什么选择 L^AT_EX

- 专业性：其生成的数学公式和排版效果达到出版级水准。
- 效率性：自动处理目录、参考文献、公式编号和交叉引用。
- 稳定性：在处理长达数百页的论文或书籍时，不会出现卡顿或格式错乱。

§ 1.2

数学公式示例

在 L^AT_EX 中，插入行内公式使用 `$ ... $`，例如： $E = mc^2$ 。

如果是独立行公式，可以使用 `$$ ... $$` 或 `equation` 环境：

$$\int_a^b f(x) dx = F(b) - F(a) \quad (1.1)$$

§ 1.3 $\text{\LaTeX}$ 的安装和配置

1.3.1 安装网址和步骤

安装网址

目前主流可能会使用 TeX Live 或 MiKTeX 作为 $\text{\LaTeX}$ 的发行版。但有的时候老师会给你发 CTeX。（因为我国国际大学生物理学术竞赛的时候老师就给我发的这玩意）

别用 CTeX！因为这玩意已经不更新了！而且它的安装包里包含了很多过时的软件和宏包，可能会导致兼容性问题。建议直接使用 TeX Live 或 MiKTeX，这两者都在不断更新，支持最新的 $\text{\LaTeX}$ 功能和宏包。

我自己用的 Texlive，这个安装包比较大，所用的时间比较长（就是上次下载了 20 个小时的那个玩意，但是我同学说清华源只要半个小时）

下面是清华源的地址：<https://mirrors.tuna.tsinghua.edu.cn/CTAN/systems/texlive/Images/>
直接下载那个 texlive.iso 就 OK。

安装步骤

下载完那个 .iso 文件后，使用解压软件（我是穷鬼，所以我直接用的 360 压缩）将其解压到一个文件夹中。然后右键以管理员身份运行解压后的文件夹中的安装程序 install-tl-windows.bat，按照提示进行安装。

但有的时候我们会长时间停留在 load 界面（比如我），然后我从 CSDN 上面找了个方法：

1. 启动终端：按下 Win + R 键，输入 cmd 并回车进入命令提示符。
2. 切换目录：进入 TeX Live 镜像文件解压后的文件夹（假设在 G 盘）：

```
d:  
cd G:\Texlive2025
```

3. 启动安装程序：输入以下命令以纯文本模式（非 GUI）启动安装：

```
install-tl-windows.bat --no-gui
```

4. 选择操作：

- 输入 I：按默认路径直接开始安装。
- 输入 D：进入路径设置界面。

5. 修改路径：若进入了 D 界面，输入数字 1，你会看到：

```
New value for TEXDIR [C:/texlive/2025]:
```

在冒号后输入你自定义的安装路径（例如 D:/Software/texlive/2025）。

6. 执行安装：路径修改完成后，输入 R 返回初始界面，最后输入 I 开始安装。

之后的话按照上面提示走就可以了。

1.3.2 配置 VSCode

安装好 $\text{\LaTeX}$ 之后，我们还需要一个编辑器来编写 $\text{\LaTeX}$ 文档。（就像我们写 C++ 或者 Python 需要一个 IDE 一样）Visual Studio Code（VSCode）是一个非常流行的代码编辑器，支持多种编程语言和扩展，非常适合用来编写 $\text{\LaTeX}$ 文档。

这是 VSCode 的官网：<https://code.visualstudio.com/>

直接下载那个 Stable 版本就 OK。（郁琰学长好像已经帮你下载过了？）

安装好 VSCode 之后，我们会发现居然是英文界面，吓死我了！不过不用担心，VSCode 支持多语言界面，我们可以轻松切换到中文界面。下载一个插件就行，在左边那个三个方块组成的图标（扩展）里搜索 **Chinese (Simplified) Language Pack for Visual Studio Code**，安装后重启 VSCode 就可以了。

我们需要安装一个 $\text{\LaTeX}$ 插件来支持 $\text{\LaTeX}$ 的编写和编译。推荐使用 **LaTeX Workshop** 插件，它提供了丰富的功能，如语法高亮、自动补全、错误检查和 PDF 预览。本人亲测十分好用！

下一步就是配置 $\text{\LaTeX}$ 的编译器路径了。打开 VSCode 的设置（File -> Preferences -> Settings），搜索 `latex-workshop.latex.tools`，找到 `latex-workshop.latex.tools` 配置项，点击编辑（Edit in settings.json），在里面添加以下内容：

```
{
  //%***** LaTeX Workshop 核心配置
  *****%//
  "latex-workshop.showContextMenu": true,
  "latex-workshop.intellisense.package.enabled": true,
  "latex-workshop.latex.build.forceRecipeUsage": false,

  //%***** 编译指令 (Recipes)
  *****%//
  "latex-workshop.latex.recipes": [
    {
      "name": "xelatex",
      "tools": ["xelatex"]
    },
    {
      "name": "pdflatex",
      "tools": ["pdflatex"]
    },
    {
      "name": "xelatex -> bibtex -> xelatex*2",
      "tools": ["xelatex", "bibtex", "xelatex", "xelatex"]
    }
  ]
}
```

```

        "name": "pdflatex -> bibtex -> pdflatex*2",
        "tools": ["pdflatex", "bibtex", "pdflatex", "pdflatex"]
    }
],

//%***** 编译工具具体参数 (Tools)
*****%//
"latex-workshop.latex.tools": [
    {
        "name": "xelatex",
        "command": "xelatex",
        "args": [
            "-shell-escape",
            "-synctex=1",
            "-interaction=nonstopmode",
            "-file-line-error",
            "%DOCFILE%"
        ]
    },
    {
        "name": "pdflatex",
        "command": "pdflatex",
        "args": [
            "-shell-escape",
            "-synctex=1",
            "-interaction=nonstopmode",
            "-file-line-error",
            "%DOCFILE%"
        ]
    },
    {
        "name": "bibtex",
        "command": "bibtex",
        "args": ["%DOCFILE%"]
    }
],

//%***** 预览与正反搜索配置 (内置查看器方案)
*****%//
// 使用 VS Code 内置标签页查看 PDF, 不再依赖外部程序
"latex-workshop.view.pdf.viewer": "tab",

// 设置双击 PDF 即可跳回对应的代码行 (反向搜索)
"latex-workshop.view.pdf.internal.synctex.keybinding": "double-click",

```



```
// 每次保存文件时自动编译
"latex-workshop.latex.autoBuild.run": "onSave",

//%***** 辅助配置
*****%/
"latex-workshop.latex.clean.fileTypes": [
  "*.aux", "*.bbl", "*.blg", "*.idx", "*.ind", "*.lof", "*.lot",
  "*.out", "*.toc", "*.acn", "*.acr", "*.alg", "*.glg", "*.glo",
  "*.gls", "*.ist", "*.fls", "*.log", "*.fdb_latexmk"
],
"security.workspace.trust.untrustedFiles": "open",
"explorer.confirmDelete": false
}
```

其中 `command` 字段需要根据你安装的 L^AT_EX 发行版进行调整。如果你使用的是 TeX Live, 通常命令就是 `xelatex` 和 `pdflatex`。如果你使用的是 MiKTeX, 可能需要指定完整路径, 例如 `C:/Program Files/MiKTeX/miktex/bin/xelatex.exe`。配置完成后, 保存 `settings.json` 文件。现在你就可以在 VSCode 中编写 L^AT_EX 文档了! 按下 `Ctrl + Alt + B` 就可以编译当前文档并预览生成的 PDF 了。

或者不想那么麻烦配置的话可以考虑下载 `Texstudio` 这个集成开发环境, 它自带了编译器和预览功能, 安装后就可以直接使用了。不过我个人觉得 VSCode 的插件生态更丰富, 使用起来也更灵活一些。下载官网 <https://texstudio.sourceforge.net/>

你好，小顾！

这一节我想和你聊聊关于 $\text{\LaTeX}$ 的一些基础的知识，就相当于我们学 C++ 的时候老师会让我们打出一个“Hello,World!”程序一样。之前看一本讲 Python 的书，里面说写“Hello,World!”具有某种使编写的程序不报错的神奇作用（程序员的神奇迷信 hhhh）

闲话不多说，我们来看看 $\text{\LaTeX}$ 的基本结构和编译流程吧！然后我们将尝试撰写第一个简单的 $\text{\LaTeX}$ 文档，看看它是如何从一个 .tex 文件变成一个漂亮的 .pdf 文件的。

§2.1

 $\text{\LaTeX}$ 文档的基本结构 (Preamble vs. Body)

$\text{\LaTeX}$ 文档由两大部分组成，分工明确：

- 导言区 (**Preamble**): 位于 `\documentclass` 和 `\begin{document}` 之间。用于加载宏包、定义全局格式（如页边距、字体）和设置元数据（标题、作者）。
- 正文区 (**Body**): 位于 `document` 环境内部。这是你输入实际文字、图表和公式的地方。

下面我们来详细聊聊每个部分

2.1.1 文档类声明: `documentclass`

每个 $\text{\LaTeX}$ 文档都必须以 `documentclass` 开头。这一命令用于定义文档的类型，也就是我们要创建的文档格式，比如文章、报告、书籍等。

```
\documentclass[选项]{文档类}
```

常见的文档类

`article` / `ctexart` **轻量化选择**。适合论文、作业。没有“章”(`\chapter`)的概念。

`report` / `ctexrep` **结构化选择**。适合较长的实验报告或本科毕业论文，支持“章”。

`book` / `ctexbook` **出版级选择**。默认双面打印，会自动处理奇偶页的页码位置。

`beamer` **演示神器**。用于制作 PDF 格式的幻灯片，学术会议必备。

常见选项

10pt,11pt,12pt 设置字体大小，默认为 10pt.

a4paper,letterpaper 设置纸张大小，默认为 letterpaper

twocolumn 将文档排版为双栏格式

draft 启用调试，显示超出边界的内容

如

```
\documentclass[12pt, a4paper]{article}
```

上述示例表示创建一个字体大小为 12pt、纸张为 A4 的文章文档。

这样看你可能觉得有点抽象，那么我们可以看看最后的那个演示代码，我会给你详细的注释，这边做个了解就好。

§2.2

宏包 (Package) 的使用与管理

2.2.1 加载宏包

宏包是 LaTeX 的扩展功能模块，用于实现特定的功能。例如：

```
\usepackage{graphicx} % 加载图片支持
\usepackage{amsmath}  % 数学公式支持
\usepackage{ctex}     % 中文支持
```

2.2.2 全局设置

在导言区可以设置文档的全局属性，例如页边距、字体、颜色等。例如：

```
\setlength{\parindent}{2em} % 设置段落缩进为 2 个字符
\setlength{\parskip}{1em}   % 设置段落间距为 1em
```

§2.3 第一个 $\text{\LaTeX}$ 文档示例

下面就是一个标准的 $\text{\LaTeX}$ 源代码示例。麻雀虽小，五脏俱全。你可以直接把这个复制粘贴到你的 VS Code 里，保存为 `hello_gu.tex`，然后用 $\text{\XeLaTeX}$ 编译，就能看到你的第一个 $\text{\LaTeX}$ 文档了！

Listing 2.1: Hello Gu 完整代码

```
% ===== 1. 导言区 (Preamble) =====  
\documentclass{ctexart} % 使用支持中文的文档类  
  
% ===== 2. 正文区 (Body) =====  
\begin{document}  
  
Hello, Gu!  
你好，小顾！  
  
\end{document}
```

代码解析：

- `ctexart`: 是专门为中文优化的文档类，确保你的“Gu”能被正确识别。
- `document` 环境: 只有放在这两个命令之间的内容才会被打印到 PDF 上。

基础排版语法

好的！相信你已经成功输出了第一个.pdf 文件。而 $\text{\LaTeX}$ 作为一个排版工具，我们来看看他是怎么进行排版的。虽然我们正常使用的时候会直接套用模板（你如果要我会发给你的，嗯），但是了解一些基础的排版语法对于后续的文档编写和个性化调整非常有帮助。好的，下面我们一起来看看吧！

§3.1

标题与分级结构

在 $\text{\LaTeX}$ 中，良好的文档结构能极大提升阅读体验和排版效果。

3.1.1 基本标题命令

$\text{\LaTeX}$ 提供了一系列命令用于创建文档的分级结构，常见的有：

chapter：章节标题

section：生成一级标题。在 **article** 文档中一般用于章节分隔；在 **report** 或 **book** 中会与 **chapter** 配合使用。

subsection：生成二级标题，用于细分章节内容。

subsubsection：生成三级标题，适合对部分内容进行更详细划分。

paragraph 与 **subparagraph**：用于进一步划分段落，这两个命令通常格式较为紧凑，默认不自动编号。示例：

```
\documentclass{article}
\usepackage[utf8]{inputenc} % 编码设置

\begin{document}

    \tableofcontents % 自动生成目录

    \section{引言}
    \lipsum[1]

    \subsection{研究背景}
```



```

\lipsum[2]

\subsubsection{问题定义}
\lipsum[3]

\paragraph{小结: } 这里简要说明了引言部分的核心问题。
\lipsum[4]

\section{方法介绍}
\lipsum[5]

\end{document}

```

3.1.2 无编号标题

在某些场合（例如前言、致谢、附录），我们可能不希望标题带有自动编号。这时只需在标题命令后加上星号*：

```
\section*{前言}：生成无编号的一级标题。
```

3.1.3 标题格式定制

嗯，我常常会觉得默认的标题丑丑的.....

所以我们来看看怎么修改标题的格式吧！可以使用 `titlesec` 等宏包来自定义标题格式，例如调整字体、字号、颜色、编号格式及间距等。下面是一个示例：

```

\documentclass{article}
\usepackage[utf8]{inputenc}
\usepackage{titlesec}
\usepackage{xcolor}

% 定制一级标题格式：红色、粗体、较大字号，编号后加冒号
\titleformat{\section}
{\color{red}\bfseries\Large} % 格式设置：红色、粗体、Large 字号
{\thesection:\quad} % 编号格式：显示章节编号后跟冒号和间距
{0pt} % 编号与标题之间的额外间距
{} % 标题前的附加代码

\begin{document}

\tableofcontents

\section{定制标题示例}

```

这里展示了自定义格式的一级标题。通过 `\texttt{titlesec}` 宏包，你可以对标题的风格进行任意定制。

`\subsection{默认格式的二级标题}`
这里是二级标题部分的内容。

`\end{document}`

§3.2

正文输入：换行、分段与特殊字符

3.2.1 空白字符与缩进控制

在 $\text{\LaTeX}$ 中，源代码中的空格和回车处理遵循以下原则：

- 空格合并：源代码中连续的多个空格会被处理为一个空格。
- 单次垂直回车：在行末按下一次回车键， $\text{\LaTeX}$ 仅将其视为空格。
- 分段 (**Paragraph**)：连续按下两次回车（即中间留出一个空行）表示开始新的一段。也可以使用 `\par` 命令。
- 缩进控制：中文排版通常默认首行缩进。若某段不需要缩进，在该段开头使用 `\noindent`；若需强制缩进，使用 `\indent`。

3.2.2 换行、换页与间距控制

除了自然分段，有时需要人工干预排版布局：

- 强制换行：使用 `\\` 或 `\newline`。注意：强制换行后的新行不会有首行缩进。
- 禁止断行：使用波浪号 `~` 代替空格（如 `Fig.~1`），可以防止在此处发生自动换行，保持语义连贯。
- 强制换页：使用 `\clearpage` 或 `\newpage`。前者会立即处理所有未排版的浮动体（如图片、表格）并另起一页，推荐在章节结束时使用。

3.2.3 常见的页面布局（A4）

A4 纸张的物理尺寸为 $210\text{mm} \times 297\text{mm}$ 。在本模板中，我们通过 `geometry` 宏包进行了如下设置：

- 版心宽度：当前正文宽度（`\textwidth`）为 418.25368pt 。

- 弹性留白：使用 `\vfill` 可以实现垂直方向的弹性填充。

```
\usepackage{geometry}
\geometry{
  a4paper,           % 确定纸张规格
  width=150mm,       % 直接指定正文宽度
  height=240mm,      % 直接指定正文高度
  left=30mm,         % 左边距（通常为了装订留多一点）
  top=25mm,          % 上边距
  marginparsep=3mm,  % 正文与边注之间的间距
  marginparwidth=20mm, % 边注的宽度
  showframe          % 【技巧】在预览中显示布局边框，方便调试
}
```

3.2.4 段落与行间距控制

在 LaTeX 中，间距的调整通常分为“行间距”和“段间距”：

1. 行间距 (**Line Spacing**): 通常在导言区使用 `setspace` 宏包进行全局调整：

```
\usepackage{setspace}
\onehalfspacing % 1.5 倍行距
\setstretch{1.25} % 自定义倍数
```

2. 段间距 (**Paragraph Spacing**):

- 全局设置：通过修改 `\parskip` 的值。例如：`\setlength{\parskip}{0.5em}`。
- 局部调整：在两个段落之间手动插入垂直间距。
 - `\smallskip`, `\medskip`, `\bigskip`: 标准预设间距。
 - `\vspace{len}`: 自定义精确高度，如 `\vspace{10pt}`。

3. 水平间距：若需在行内手动增加空格，可使用 `\hspace{len}`。特殊的弹性间距 `\hfill` 可以将文字推向行首或行尾。

3.2.5 常用的长度单位

在设置间距时，了解 LaTeX 的单位非常重要：

- `pt`: 点 (Point), 约 0.35mm。
- `cm/mm`: 厘米/毫米。
- `em`: 相对于当前字体中字母 “M” 的宽度（推荐用于水平间距）。

- **ex**: 相对于当前字体中字母“x”的高度（推荐用于垂直间距）。

```
% --- 1. 换行与禁止断行示例 ---
\noindent 这是一行正文内容。\\
使用了双反斜杠强制换行，下一行不会有缩进。

这是一段非常长的文字，为了防止像 Fig.~1 这样的引用被
拆散在两行，我们使用了波浪号。 % Fig.~1 永远不会在 Fig. 处断开

% --- 2. 垂直间距示例 ---
\section{垂直间距对比}
第一段。
\smallskip
这是使用 \texttt{\textbackslash smallskip} 产生的间距。

\medskip
这是使用 \texttt{\textbackslash medskip} 产生的间距。

\bigskip
这是使用 \texttt{\textbackslash bigskip} 产生的间距。

\vspace{2.5ex}
这是使用 \texttt{\textbackslash vspace\{2.5ex\}} 自定义的精确间距。

% --- 3. 水平间距与弹性填充示例 ---
\section{水平排版技巧}
固定间距：A \hspace{3cm} B （中间空了 3cm）

弹性填充示例（常用于页脚或签名）： \par
\medskip
\noindent 导师签名： \underline{\hspace{3em}} \hfill 日期： \today

% --- 4. 页面单位示例 ---
\noindent \rule{0.5\textwidth}{1pt} % 画一条占正文宽度一半的水平线
```

3.2.6 特殊字符与转义

LaTeX 预留了一些具有特殊功能的字符。如果直接输入，可能会导致编译错误或排版异常：

3.2.7 注释技巧

使用 % 可以注释掉单行内容。如果你需要大段注释，建议加载 `verbatim` 宏包并使用 `comment` 环境：

字符	转义输入	功能说明
#	\#	参数引导符
\$	\\$	数学模式引导符
%	\%	注释符号（该符号后的内容不被编译）
&	\&	表格列分隔符
_	_	数学下标符号
{ }	\{ \}	环境参数界定符
~	\textasciitilde	不可断行空格
^	\textasciicircum	数学上标符号
\	\textbackslash	命令引导符

```

\begin{verbatim}
\usepackage{verbatim}
\begin{comment}
  这里的文字
  无论写多少行
  都不会出现在 PDF 中
\end{comment}
\end{verbatim}

```

§3.3

字体控制：字号、粗体、斜体与下划线

3.3.1 字号控制

你可以使用内置命令改变字号大小：

- 最小号 (tiny)
- 小号 (small)
- 标准 (normalsize)
- 大号 (large)

• 巨大 (huge)

- 使用 ctex 宏包特有命令：这是四号字，这是小四号字。

```
\begin{itemize}
  \item {\tiny 最小号 (tiny)}
  \item {\small 小号 (small)}
  \item {\normalsize 标准 (normalsize)}
  \item {\large 大号 (large)}
  \item {\huge 巨大 (huge)}
  \item 使用 ctex 宏包特有命令：\zihao{4} 这是四号字，\zihao{-4} 这是小四号字。
\end{itemize}
```

3.3.2 字体样式

这里展示常见的文本格式化方法：

- 这是粗体 (**Boldface**)
- 这是斜体 (*Italic*) —— 在中文里通常表现为楷体。
- 这是下划线 (Underline) —— 建议使用 ulem 宏包的 \uline 命令。
- 这是等宽字体 (Typewriter) —— 常用于表示代码或路径。

```
\begin{itemize}
  \item \textbf{这是粗体 (Boldface)}
  \item \textit{这是斜体 (Italic)} —— 在中文里通常表现为楷体。
  \item \uline{这是下划线 (Underline)} —— 建议使用 \texttt{ulem} 宏包的 \texttt{\textbackslash ulem} 命令。
  \item \texttt{这是等宽字体 (Typewriter)} —— 常用于表示代码或路径。
\end{itemize}
```

下面是一个字体字号的实践。。。。。

```
\documentclass{article}

% 引入必要的宏包
\usepackage{ulem} % 用于删除线
\usepackage{xcolor} % 用于文本颜色

\begin{document}

  % 标题
  \section*{LaTeX 文本格式化示例}

  % 加粗文本
  \textbf{这是加粗的文本}\n
```

```

% 斜体文本
\textit{这是斜体的文本}\\

% 下划线文本
\underline{这是带下划线的文本}\\

% 删除线文本
\sout{这是带删除线的文本}\\

% 上标和下标
这是上标:  $x^2$ , 这是下标:  $y_1$ \\

% 字体大小调整
{\tiny 这是 tiny 字体}\\
{\small 这是 small 字体}\\
{\normalsize 这是 normalsize 字体}\\
{\large 这是 large 字体}\\
{\Large 这是 Large 字体}\\
{\LARGE 这是 LARGE 字体}\\
{\huge 这是 huge 字体}\\
{\Huge 这是 Huge 字体}\\

% 文本颜色
\textcolor{red}{这是红色文本}\\
\textcolor{blue}{这是蓝色文本}\\
\textcolor{green}{这是绿色文本}\\

% 混合效果示例
\textbf{\textit{\textcolor{purple}{这是加粗+斜体+紫色的文本}}}\
\underline{\textcolor{orange}{这是带下划线+橙色的文本}}\

\end{document}

```

§3.4

列表环境：有序与无序列表

3.4.1 无序列表 (itemize)

无序列表使用项目符号：

```
\begin{itemize}
  \item 苹果
  \item 香蕉
  \item 橙子
\end{itemize}
```

- 苹果
- 香蕉
- 橙子

3.4.2 有序列表 (enumerate)

有序列表会自动生成编号：

```
\begin{enumerate}
  \item 第一步：打开冰箱门
  \item 第二步：把大象放进去
  \item 第三步：关上冰箱门
\end{enumerate}
```

1. 第一步：打开冰箱门
2. 第二步：把大象放进去
3. 第三步：关上冰箱门

hhh 这个笑话好老，我果然是老年人了哈哈哈。。

3.4.3 嵌套列表

列表是可以嵌套使用的：

```
\begin{enumerate}
  \item 编程语言
  \begin{itemize}
    \item C++
    \item Python
  \end{itemize}
  \item 标记语言
  \begin{itemize}
    \item HTML
    \item \LaTeX
  \end{itemize}
\end{enumerate}
```


1. 编程语言

- C++
- Python

2. 标记语言

- HTML
- $\text{\LaTeX}$

§3.5 插入超链接与目录

嘿嘿，有没有发现这个文档的目录点击即可跳转到相应章节？这就是一种超链接。在学术论文、技术报告和各类电子文档中，一个清晰的目录和便捷的超链接能大大提升文档的可读性和导航体验。 $\text{\LaTeX}$ 为此提供了两项强大的功能：

1. 自动生成目录：通过 `tableofcontents` 命令， $\text{\LaTeX}$ 可以扫描文档中的标题命令，并自动生成包含各级标题和页码的目录。
2. 插入超链接：使用 `hyperref` 宏包，可以将文档内的交叉引用和目录项都变成超链接，同时还能通过 `href` 命令插入指向外部网址的链接，以及利用 `hypertarget` 和 `hyperlink` 命令自定义内部跳转链接。

3.5.1 自动生成目录

基本用法

在 $\text{\LaTeX}$ 中，生成目录只需在需要显示目录的地方插入 `tableofcontents` 命令。 $\text{\LaTeX}$ 会扫描文档中所有支持的标题命令，并自动生成目录列表。示例如下：

```
\documentclass{article}
\usepackage{ctex}
\usepackage[utf8]{inputenc} % 设置文档输入编码

\begin{document}

    \tableofcontents % 生成目录

    \section{引言}
    这里介绍文档的背景和目的。

    \subsection{研究背景}
    阐述研究的理论基础与现状。
```

```

\section{方法介绍}
    详细描述实验方法和数据处理过程。

\end{document}

```

注意事项：每次修改标题后，建议至少编译两次（或使用 L^AT_EX 编辑器的“多次编译”功能），以确保目录中的页码信息正确更新。

超链接化目录

使用 `hyperref` 宏包后，生成的目录会自动变成可点击的超链接。这意味着读者可以通过点击目录中的各级标题，直接跳转到对应的章节。

3.5.2 插入超链接

为了使文档中的链接变得可点击（无论是目录、交叉引用还是手动插入的链接），我们需要加载 `hyperref` 宏包。

加载 `hyperref` 宏包

在导言区加载 `hyperref` 宏包时，可以设置一些选项来控制链接的颜色、风格等。常用的设置如下：

```

\usepackage[colorlinks=true,          % 启用彩色链接而非带框链接
linkcolor=blue,                      % 内部链接（如目录、交叉引用）颜色为蓝色
urlcolor=blue,                      % 外部链接颜色为蓝色
citecolor=blue]{hyperref} % 参考文献链接颜色为蓝色

```

将 `hyperref` 宏包加载在所有其他宏包之后，确保其能正确修改文档内的所有交叉引用和链接。

插入外部超链接

使用 `hyperref` 宏包后，可以利用 `href` 命令插入指向外部网站的超链接。基本语法为：

```

\href{<URL>}{<显示文本>}

```

插入内部链接

除了外部链接外，有时还需要在文档内部实现跳转。`hyperref` 提供了两组命令来实现内部链接：

自动链接： 使用 `tableofcontents`、`ref`、`pageref` 等命令生成的链接在加载 `hyperref` 后均自动成为超链接。

自定义内部链接: `hypertarget` 与 `hyperlink` 命令创建自定义的内部链接。

使用方法

1. 在目标位置设置锚点:

```
\hypertarget{sec:intro}{}

```

2. 在其他地方插入跳转链接:

```
请参见 \hyperlink{sec:intro}{引言} 部分。

```

综合示例

```
\documentclass{article}
\usepackage{ctex}
\usepackage[utf8]{inputenc} % 设置输入编码
\usepackage[T1]{fontenc}    % 设置输出编码, 确保字符显示正确
\usepackage{lipsum}         % 生成示例文本
\usepackage[colorlinks=true, linkcolor=blue, urlcolor=blue, citecolor=blue]{
  hyperref}

\begin{document}

  \title{综合示例: 目录与超链接}
  \author{作者姓名}
  \date{\today}
  \maketitle

  \tableofcontents % 自动生成目录

  \bigskip
  \hrule
  \bigskip

  \section{引言} \hypertarget{sec:intro}{}
  在本篇文章中, 我们将详细介绍如何使用 LaTeX 的目录功能以及插入超链接。
  请访问 \href{https://www.latex-project.org}{LaTeX 官方网站} 获取更多信息。

  \section{文档结构}
  本节描述文档结构与目录生成的基本原理。
  通过 \verb|\tableofcontents| 命令, LaTeX 会自动扫描所有标题命令, 并生成一个结构清晰的目录。
```

点击目录中的各个标题可以直接跳转到对应章节。

```
\subsection{目录的生成}
\lipsum[1] % 生成一些占位文本

\subsection{超链接的应用}
在本文中，超链接主要分为以下两种：
\begin{itemize}
  \item \textbf{外部超链接}：例如指向 \href{https://www.latex-
    project.org}{LaTeX 官方网站};
  \item \textbf{内部超链接}：例如点击 \hyperlink{sec:intro}{这
    里} 可跳转回引言部分。
\end{itemize}

\section{结论}
通过本示例，我们了解了如何：
\begin{enumerate}
  \item 使用 \verb|\tableofcontents| 自动生成目录，并实现目录项
    的超链接功能；
  \item 利用 hyperref 宏包，通过 \verb|\href| 插入外部链接；
  \item 使用 \verb|\hypertarget| 与 \verb|\hyperlink| 实现自定义
    内部链接，增强文档导航体验。
\end{enumerate}

\bigskip
\hrule
\bigskip

\noindent
\textbf{Hello \LaTeX}

\end{document}
```

§3.6 插入脚注

在学术论文、技术文档及各类写作中，脚注和注释是常见的排版工具，用于提供额外的信息、引用来源或解释概念。我觉得使用 Word 进行脚注是个非常麻烦的事情，但是这件事在 $\text{\LaTeX}$ 中十分简单便捷。我们一起来看看吧！

3.6.1 插入脚注

脚注常用于文献引用、进一步说明或给出额外信息。在 LaTeX 中，插入脚注非常简单，通过

```
\footnote
```

命令即可。

基本用法

脚注的基本语法如下：

```
\footnote{这里是脚注内容}
```

当你使用脚注时，LaTeX 会自动为脚注内容编号，并把脚注内容放在页面底部。

多个脚注

在同一个段落或文本中，你可以使用多个脚注，LaTeX 会自动为每个脚注编号。好方便！

脚注与章节标题的兼容

在有些文档中，我们需要在章节标题中插入脚注。LaTeX 默认允许这样做，并会将脚注编号与标题内容一起显示。

§4.1 行内公式与块级公式

LaTeX 强大的数学排版功能是其最受欢迎的特性之一。在数学模式中，公式可以分为两大类：行内公式和块级公式。

4.1.1 行内公式

行内公式用于在正文中嵌入较短的数学表达式，通常用来书写变量、简单公式或符号。常用的写法有两种：

美元符号法：使用一对美元符号 `$...$` 将数学表达式包裹起来。

圆括号法：使用

$$\backslash(. . .\backslash)$$

同样可以达到行内公式的效果，通常这种方式更具可读性。

在行内公式中，LaTeX 会自动调整字体和间距，使得数学表达式与周围文本保持协调。但如果公式过长或复杂，则建议使用块级公式，以避免影响正文排版。

4.1.2 块级公式

块级公式用于单独展示较长或较重要的数学表达式，通常居中显示，并且可以自动编号（或取消编号）。常见的写法有以下几种：

1. 使用

$$\backslash[. . .\backslash]$$

这是最简单的块级公式写法，不会产生公式编号，适用于无需引用的公式。

2. 使用 `equation` 环境

该环境适用于需要编号的公式，方便后续引用。使用前记得引入 `amsmath` 包。

```
\begin{equation}
  \label{eq:quadratic}
  ax^2+bx+c=0
\end{equation}
```

可以在文中通过

```
\eqref{eq:quadratic}
```

来引用该公式。

3. 使用 `align` 环境对于多个相关联的公式或者需要对齐等号的情况，`align` 环境非常实用。同样需要加载 `amsmath` 包。

§4.2 数学符号与运算符

见本章最后。

```
% 行内公式中使用希腊字母
设  $\alpha$ ,  $\beta$ ,  $\gamma$  为角度，而  $\Gamma(n)$  表示伽马函数。
```

4.2.1 上下标

上下标是数学表达式中常见的元素，LaTeX 提供了简单的语法来实现它们。使用 `^` 来表示上标，使用 `_` 来表示下标。例如：

```
% 上标和下标示例
设  $x_i$  表示第  $i$  个数据点，而  $y^2$  表示  $y$  的平方。
```

4.2.2 分数与根号

分数可以使用 `\frac` 命令来表示，语法为 `\frac 分子分母`。例如：

```
% 分数示例
设  $\frac{a}{b}$  表示  $a$  除以  $b$ 。
```

连

分数可以使用 `\cfrac` 命令来表示，语法为 `\cfrac` 分子分母。例如：

% 连分数示例

设 `\cfrac{1}{2 + \cfrac{1}{3}}` 表示一个连分数。

或者直接分数嵌套使用。

根号可以使用 `\sqrt` 命令来表示，语法为 `\sqrt` 被开方数。例如：

% 根号示例

设 `\sqrt{x}` 表示 x 的平方根。

§4.3 矩阵、多行方程组与编号控制

下面是一个带编号的多行对齐方程组（使用 `&` 对齐等号）：

$$\begin{aligned}(a+b)^2 &= a^2 + 2ab + b^2 \\ (a-b)^2 &= a^2 - 2ab + b^2\end{aligned}\tag{4.1}$$

如果是排版方括号矩阵，可以使用 `bmatrix` 环境：

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}\tag{4.2}$$

Listing 4.1: 矩阵排版源代码参考

```
\begin{bmatrix}
  a_{11} & a_{12} \\
  a_{21} & a_{22}
\end{bmatrix}
```

如果是排版圆括号矩阵，可以使用 `pmatrix` 环境：

$$B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}\tag{4.3}$$

还有范数符号（双竖线），使用 `Vmatrix` 来表示：

```
\lVert v \rVert = \begin{Vmatrix}
  x \\
  y \\
  z
\end{Vmatrix}
```


§4.4

妙招

依旧如果我们不适应这种直接敲公式的方法,我还有小妙招!<https://www.latexlive.com/home> 这个网站可以把公式转换为 tex, 或者 Mathtype (Word 里面的一个插件, 小顾你以后一定会用) 中直接复制出来, 就是 LaTeX 格式, 其实很方便。

§4.5

符号大全

这是我常用的一个符号查找的表。是我用 Markdown 的时候用的。Markdown 和 L^AT_EX 的符号是一样的。

1 Greek and Hebrew letters

α	<code>\alpha</code>	κ	<code>\kappa</code>	ψ	<code>\psi</code>	$\digamma$	<code>\digamma</code>	Δ	<code>\Delta</code>	Θ	<code>\Theta</code>
β	<code>\beta</code>	λ	<code>\lambda</code>	ρ	<code>\rho</code>	ε	<code>\varepsilon</code>	Γ	<code>\Gamma</code>	Υ	<code>\Upsilon</code>
χ	<code>\chi</code>	μ	<code>\mu</code>	σ	<code>\sigma</code>	$\varkappa$	<code>\varkappa</code>	Λ	<code>\Lambda</code>	Ξ	<code>\Xi</code>
δ	<code>\delta</code>	ν	<code>\nu</code>	τ	<code>\tau</code>	φ	<code>\varphi</code>	Ω	<code>\Omega</code>		
ϵ	<code>\epsilon</code>	$\omicron$	<code>\omicron</code>	θ	<code>\theta</code>	ϖ	<code>\varpi</code>	Φ	<code>\Phi</code>	$\aleph$	<code>\aleph</code>
η	<code>\eta</code>	ω	<code>\omega</code>	υ	<code>\upsilon</code>	ϱ	<code>\varrho</code>	Π	<code>\Pi</code>	$\beth$	<code>\beth</code>
γ	<code>\gamma</code>	ϕ	<code>\phi</code>	ξ	<code>\xi</code>	ς	<code>\varsigma</code>	Ψ	<code>\Psi</code>	$\gimel$	<code>\gimel</code>
ι	<code>\iota</code>	π	<code>\pi</code>	ζ	<code>\zeta</code>	ϑ	<code>\vartheta</code>	Σ	<code>\Sigma</code>		

图 4.1: 希腊字母和希伯来字母表

2 L^AT_EX math constructs

$\frac{abc}{xyz}$	<code>\frac{abc}{xyz}</code>	$\overline{abc}$	<code>\overline{abc}</code>	$\overrightarrow{abc}$	<code>\overrightarrow{abc}</code>
f'	<code>f'</code>	$\underline{abc}$	<code>\underline{abc}</code>	$\overleftarrow{abc}$	<code>\overleftarrow{abc}</code>
$\sqrt{abc}$	<code>\sqrt{abc}</code>	$\widehat{abc}$	<code>\widehat{abc}</code>	$\overleftrightarrow{abc}$	<code>\overleftrightarrow{abc}</code>
$\sqrt[n]{abc}$	<code>\sqrt[n]{abc}</code>	$\widetilde{abc}$	<code>\widetilde{abc}</code>	$\underbrace{abc}$	<code>\underbrace{abc}</code>

图 4.2: LaTeX 数学表达式

3 Delimiters

$ $	<code> </code>	$\{$	<code>\{</code>	$\lfloor$	<code>\lfloor</code>	$\uparrow$	<code>\Uparrow</code>	$\llcorner$	<code>\llcorner</code>
$\ $	<code>\ </code>	$\}$	<code>\}</code>	$\rfloor$	<code>\rfloor</code>	$\uparrow$	<code>\uparrow</code>	$\lrcorner$	<code>\lrcorner</code>
$\ $	<code>\ </code>	$\langle$	<code>\langle</code>	$\lceil$	<code>\lceil</code>	$\downarrow$	<code>\Downarrow</code>	$\ulcorner$	<code>\ulcorner</code>
$\ $	<code>\ </code>	$\rangle$	<code>\rangle</code>	$\rceil$	<code>\rceil</code>	$\downarrow$	<code>\downarrow</code>	$\urcorner$	<code>\urcorner</code>

图 4.3: 分隔符和箭头符号

4 Variable-sized symbols (displayed formulae show larger version)

$\sum$	<code>\sum</code>	$\int$	<code>\int</code>	$\biguplus$	<code>\biguplus</code>	$\bigoplus$	<code>\bigoplus</code>	$\bigvee$	<code>\bigvee</code>
$\prod$	<code>\prod</code>	$\oint$	<code>\oint</code>	$\bigcap$	<code>\bigcap</code>	$\bigotimes$	<code>\bigotimes</code>	$\bigwedge$	<code>\bigwedge</code>
$\coprod$	<code>\coprod</code>	$\iint$	<code>\iint</code>	$\bigcup$	<code>\bigcup</code>	$\bigodot$	<code>\bigodot</code>	$\bigsqcup$	<code>\bigsqcup</code>

图 4.4: 可变长度符号

5 Standard Function Names

Function names should appear in Roman, not Italic, e.g.,

Correct: `\tan(at-n\pi)` $\rightarrow \tan(at - n\pi)$
 Incorrect: `\tan(at-n\pi)` $\rightarrow \tan(at - n\pi)$

<code>arccos</code>	<code>\arccos</code>	<code>arcsin</code>	<code>\arcsin</code>	<code>arctan</code>	<code>\arctan</code>	<code>arg</code>	<code>\arg</code>
<code>cos</code>	<code>\cos</code>	<code>cosh</code>	<code>\cosh</code>	<code>cot</code>	<code>\cot</code>	<code>coth</code>	<code>\coth</code>
<code>csc</code>	<code>\csc</code>	<code>deg</code>	<code>\deg</code>	<code>det</code>	<code>\det</code>	<code>dim</code>	<code>\dim</code>
<code>exp</code>	<code>\exp</code>	<code>gcd</code>	<code>\gcd</code>	<code>hom</code>	<code>\hom</code>	<code>inf</code>	<code>\inf</code>
<code>ker</code>	<code>\ker</code>	<code>lg</code>	<code>\lg</code>	<code>lim</code>	<code>\lim</code>	<code>lim inf</code>	<code>\liminf</code>
<code>lim sup</code>	<code>\limsup</code>	<code>ln</code>	<code>\ln</code>	<code>log</code>	<code>\log</code>	<code>max</code>	<code>\max</code>
<code>min</code>	<code>\min</code>	<code>Pr</code>	<code>\Pr</code>	<code>sec</code>	<code>\sec</code>	<code>sin</code>	<code>\sin</code>
<code>sinh</code>	<code>\sinh</code>	<code>sup</code>	<code>\sup</code>	<code>tan</code>	<code>\tan</code>	<code>tanh</code>	<code>\tanh</code>

图 4.5: 标准函数名

6 Binary Operation/Relation Symbols

$*$	<code>\ast</code>	$\pm$	<code>\pm</code>	$\cap$	<code>\cap</code>	$\triangleleft$	<code>\lhd</code>
$\star$	<code>\star</code>	$\mp$	<code>\mp</code>	$\cup$	<code>\cup</code>	$\triangleright$	<code>\rhd</code>
$\cdot$	<code>\cdot</code>	$\amalg$	<code>\amalg</code>	$\uplus$	<code>\uplus</code>	$\triangleleft$	<code>\triangleleft</code>
$\circ$	<code>\circ</code>	$\odot$	<code>\odot</code>	$\sqcap$	<code>\sqcap</code>	$\triangleright$	<code>\triangleright</code>
$\bullet$	<code>\bullet</code>	$\ominus$	<code>\ominus</code>	$\sqcup$	<code>\sqcup</code>	$\triangleleft$	<code>\triangleleft</code>
$\bigcirc$	<code>\bigcirc</code>	$\oplus$	<code>\oplus</code>	$\wedge$	<code>\wedge</code>	$\triangleright$	<code>\triangleright</code>
$\diamond$	<code>\diamond</code>	$\oslash$	<code>\oslash</code>	$\vee$	<code>\vee</code>	∇	<code>\nabla</code>
$\times$	<code>\times</code>	$\otimes$	<code>\otimes</code>	$\dagger$	<code>\dagger</code>	Δ	<code>\Delta</code>
$\div$	<code>\div</code>	$\wr$	<code>\wr</code>	$\ddagger$	<code>\ddagger</code>	$\setminus$	<code>\setminus</code>
$\cdot$	<code>\centerdot</code>	$\Box$	<code>\Box</code>	$\bar{\wedge}$	<code>\bar{\wedge}</code>	$\veebar$	<code>\veebar</code>
$\circledast$	<code>\circledast</code>	$\boxplus$	<code>\boxplus</code>	$\curlywedge$	<code>\curlywedge</code>	$\curlyvee$	<code>\curlyvee</code>
$\circledcirc$	<code>\circledcirc</code>	$\boxminus$	<code>\boxminus</code>	$\Cap$	<code>\Cap</code>	$\Cup$	<code>\Cup</code>
$\circledR$	<code>\circledR</code>	$\boxtimes$	<code>\boxtimes</code>	$\bot$	<code>\bot</code>	$\top$	<code>\top</code>
$\dot{+}$	<code>\dotplus</code>	$\boxdot$	<code>\boxdot</code>	$\intercal$	<code>\intercal</code>	$\times$	<code>\times</code>
$\ast$	<code>\divideontimes</code>	$\square$	<code>\square</code>	$\bar{\bar{\wedge}}$	<code>\doublebarwedge</code>	$\times$	<code>\times</code>
$\equiv$	<code>\equiv</code>	$\leq$	<code>\leq</code>	$\geq$	<code>\geq</code>	$\perp$	<code>\perp</code>
$\cong$	<code>\cong</code>	$\prec$	<code>\prec</code>	$\succ$	<code>\succ</code>	$ $	<code>\mid</code>
$\neq$	<code>\neq</code>	$\preceq$	<code>\preceq</code>	$\succeq$	<code>\succeq</code>	$\parallel$	<code>\parallel</code>
$\sim$	<code>\sim</code>	$\ll$	<code>\ll</code>	$\gg$	<code>\gg</code>	$\bowtie$	<code>\bowtie</code>
$\simeq$	<code>\simeq</code>	$\subset$	<code>\subset</code>	$\supset$	<code>\supset</code>	$\Join$	<code>\Join</code>
$\approx$	<code>\approx</code>	$\subseteq$	<code>\subseteq</code>	$\supseteq$	<code>\supseteq</code>	$\ltimes$	<code>\ltimes</code>
$\asymp$	<code>\asymp</code>	$\sqsubset$	<code>\sqsubset</code>	$\sqsupset$	<code>\sqsupset</code>	$\rtimes$	<code>\rtimes</code>
$\doteq$	<code>\doteq</code>	$\sqsubseteq$	<code>\sqsubseteq</code>	$\sqsupseteq$	<code>\sqsupseteq</code>	$\smile$	<code>\smile</code>
$\propto$	<code>\propto</code>	$\dashv$	<code>\dashv</code>	$\vdash$	<code>\vdash</code>	$\frown$	<code>\frown</code>
$\models$	<code>\models</code>	$\in$	<code>\in</code>	$\ni$	<code>\ni</code>	$\notin$	<code>\notin</code>

图 4.6: 二元运算符和关系运算符

7 Arrow symbols

$\leftarrow$	<code>\leftarrow</code>	$\longleftarrow$	<code>\longleftarrow</code>	$\uparrow$	<code>\uparrow</code>
$\Leftarrow$	<code>\Leftarrow</code>	$\Longleftarrow$	<code>\Longleftarrow</code>	$\Uparrow$	<code>\Uparrow</code>
$\rightarrow$	<code>\rightarrow</code>	$\longrightarrow$	<code>\longrightarrow</code>	$\downarrow$	<code>\downarrow</code>
$\Rightarrow$	<code>\Rightarrow</code>	$\Longrightarrow$	<code>\Longrightarrow</code>	$\Downarrow$	<code>\Downarrow</code>
$\leftrightarrow$	<code>\leftrightarrow</code>	$\longleftrightarrow$	<code>\longleftrightarrow</code>	$\updownarrow$	<code>\updownarrow</code>
$\Leftrightarrow$	<code>\Leftrightarrow</code>	$\Longleftrightarrow$	<code>\Longleftrightarrow</code>	$\Updownarrow$	<code>\Updownarrow</code>
$\mapsto$	<code>\mapsto</code>	$\longmapsto$	<code>\longmapsto</code>	$\nearrow$	<code>\nearrow</code>
$\hookrightarrow$	<code>\hookrightarrow</code>	$\hookrightarrow$	<code>\hookrightarrow</code>	$\searrow$	<code>\searrow</code>
$\leftharpoonup$	<code>\leftharpoonup</code>	$\rightharpoonup$	<code>\rightharpoonup</code>	$\swarrow$	<code>\swarrow</code>
$\leftharpoondown$	<code>\leftharpoondown</code>	$\rightharpoondown$	<code>\rightharpoondown</code>	$\nwarrow$	<code>\nwarrow</code>
$\rightleftharpoons$	<code>\rightleftharpoons</code>	$\leadsto$	<code>\leadsto</code>		
$\dashrightarrow$	<code>\dashrightarrow</code>	$\dashleftarrow$	<code>\dashleftarrow</code>	$\leftrightsquigarrow$	<code>\leftrightsquigarrow</code>
$\leftrightarrows$	<code>\leftrightarrows</code>	$\Lleftarrow$	<code>\Lleftarrow</code>	$\rightleftarrows$	<code>\rightleftarrows</code>
$\leftarrowtail$	<code>\leftarrowtail</code>	$\looparrowleft$	<code>\looparrowleft</code>	$\twoheadleftarrow$	<code>\twoheadleftarrow</code>
$\curvearrowleft$	<code>\curvearrowleft</code>	$\circlearrowleft$	<code>\circlearrowleft</code>	$\leftrightharpoons$	<code>\leftrightharpoons</code>
$\Uparrow$	<code>\Uparrow</code>	$\upharpoonleft$	<code>\upharpoonleft</code>	$\Lsh$	<code>\Lsh</code>
$\multimap$	<code>\multimap</code>	$\leftrightsquigarrow$	<code>\leftrightsquigarrow</code>	$\downharpoonleft$	<code>\downharpoonleft</code>
$\rightleftarrows$	<code>\rightleftarrows</code>	$\rightleftarrows$	<code>\rightleftarrows</code>	$\Rightarrow$	<code>\Rightarrow</code>
$\twoheadrightarrow$	<code>\twoheadrightarrow</code>	$\rightarrowtail$	<code>\rightarrowtail</code>	$\rightleftarrows$	<code>\rightleftarrows</code>
$\rightleftharpoons$	<code>\rightleftharpoons</code>	$\curvearrowright$	<code>\curvearrowright</code>	$\looparrowright$	<code>\looparrowright</code>
$\Rsh$	<code>\Rsh</code>	$\downdownarrows$	<code>\downdownarrows</code>	$\circlearrowright$	<code>\circlearrowright</code>
$\downharpoonright$	<code>\downharpoonright</code>	$\rightsquigarrow$	<code>\rightsquigarrow</code>	$\upharpoonright$	<code>\upharpoonright</code>
$\nleftarrow$	<code>\nleftarrow</code>	$\nrightarrow$	<code>\nrightarrow</code>	$\nleftarrow$	<code>\nleftarrow</code>
$\nrightarrow$	<code>\nrightarrow</code>	$\nleftrightarrow$	<code>\nleftrightarrow</code>	$\nrightarrow$	<code>\nrightarrow</code>

图 4.7: 箭头符号和其他符号

当然 Texshop 本身就提供了一个快速插入符号的符号表。就在 VSCode 的侧边栏里。点击 LaTeX Workshop 的图标就可以看到符号表了。

8 Miscellaneous symbols

∞	<code>\infty</code>	$\forall$	<code>\forall</code>	$\mathbb{k}$	<code>\Bbbk</code>	$\wp$	<code>\wp</code>
∇	<code>\nabla</code>	$\exists$	<code>\exists</code>	$\star$	<code>\bigstar</code>	$\angle$	<code>\angle</code>
∂	<code>\partial</code>	$\nexists$	<code>\nexists</code>	$\diagdown$	<code>\diagdown</code>	$\sphericalangle$	<code>\measuredangle</code>
$\emptyset$	<code>\emptyset</code>	$\emptyset$	<code>\emptyset</code>	$\diagup$	<code>\diagup</code>	$\sphericalangle$	<code>\sphericalangle</code>
$\clubsuit$	<code>\clubsuit</code>	$\varnothing$	<code>\varnothing</code>	$\Diamond$	<code>\Diamond</code>	$\complement$	<code>\complement</code>
$\diamond$	<code>\diamondsuit</code>	$\imath$	<code>\imath</code>	$\Finv$	<code>\Finv</code>	$\triangledown$	<code>\triangledown</code>
$\heartsuit$	<code>\heartsuit</code>	j	<code>\jmath</code>	$\Game$	<code>\Game</code>	$\triangle$	<code>\triangle</code>
$\spadesuit$	<code>\spadesuit</code>	ℓ	<code>\ell</code>	$\hbar$	<code>\hbar</code>	$\vartriangle$	<code>\vartriangle</code>
$\cdots$	<code>\cdots</code>	$\iiint$	<code>\iiint</code>	$\hslash$	<code>\hslash</code>	$\blacklozenge$	<code>\blacklozenge</code>
$\vdots$	<code>\vdots</code>	$\iiint$	<code>\iiint</code>	$\lozenge$	<code>\lozenge</code>	$\blacksquare$	<code>\blacksquare</code>
$\ldots$	<code>\ldots</code>	$\iint$	<code>\iint</code>	$\mho$	<code>\mho</code>	$\blacktriangle$	<code>\blacktriangle</code>
$\ddots$	<code>\ddots</code>	$\sharp$	<code>\sharp</code>	$\prime$	<code>\prime</code>	$\blacktriangledown$	<code>\blacktriangledown</code>
$\Im$	<code>\Im</code>	$\flat$	<code>\flat</code>	$\square$	<code>\square</code>	$\backprime$	<code>\backprime</code>
$\Re$	<code>\Re</code>	$\natural$	<code>\natural</code>	$\surd$	<code>\surd</code>	$\circledS$	<code>\circledS</code>

图 4.8: 杂项符号

§5.1

基础表格 (tabular) 与三线表 (booktabs)

在 $\text{\LaTeX}$ 中，排版表格主要使用 `tabular` 环境。三线表通过 `booktabs` 宏包提供的水平线提升专业感。

Listing 5.1: 三线表源码

```
\begin{table}[H]
  \centering
  \caption{三线表样式演示}
  \begin{tabular}{ccc}
    \toprule
    \textbf{实验组} & \textbf{参数 A} & \textbf{结果 B} \\
    \midrule
    Control & 1.00 & 0.85 \\
    \bottomrule
  \end{tabular}
\end{table}
```

表 5.1: 三线表样式演示

实验组	参数 A	结果 B
Control	1.00	0.85
Group 1	2.50	1.12
Group 2	5.00	2.04

TeXStudio 中有关于直接插入表格的按钮，但我更喜欢用 <https://www.latex-tables.com/>，可以直接把表格转换成 $\text{\LaTeX}$ 代码，很方便。

§5.2

图片插入与路径管理

图片插入的核心命令是 `\includegraphics`。建议将图片统一放在 `Figure/` 文件夹下。

1 Greek and Hebrew letters

α	<code>\alpha</code>	κ	<code>\kappa</code>	ψ	<code>\psi</code>	$\digamma$	<code>\digamma</code>	Δ	<code>\Delta</code>	Θ	<code>\Theta</code>
β	<code>\beta</code>	λ	<code>\lambda</code>	ρ	<code>\rho</code>	ε	<code>\varepsilon</code>	Γ	<code>\Gamma</code>	Υ	<code>\Upsilon</code>
χ	<code>\chi</code>	μ	<code>\mu</code>	σ	<code>\sigma</code>	$\varkappa$	<code>\varkappa</code>	Λ	<code>\Lambda</code>	Ξ	<code>\Xi</code>
δ	<code>\delta</code>	ν	<code>\nu</code>	τ	<code>\tau</code>	φ	<code>\varphi</code>	Ω	<code>\Omega</code>		
ϵ	<code>\epsilon</code>	$\omicron$	<code>\omicron</code>	θ	<code>\theta</code>	ϖ	<code>\varpi</code>	Φ	<code>\Phi</code>	$\aleph$	<code>\aleph</code>
η	<code>\eta</code>	ω	<code>\omega</code>	υ	<code>\upsilon</code>	ϱ	<code>\varrho</code>	Π	<code>\Pi</code>	$\beth$	<code>\beth</code>
γ	<code>\gamma</code>	ϕ	<code>\phi</code>	ξ	<code>\xi</code>	ς	<code>\varsigma</code>	Ψ	<code>\Psi</code>	$\aleph$	<code>\aleph</code>
ι	<code>\iota</code>	π	<code>\pi</code>	ζ	<code>\zeta</code>	ϑ	<code>\vartheta</code>	Σ	<code>\Sigma</code>	$\gimel$	<code>\gimel</code>

图 5.1: 这是你的第一张独立图片

§5.3

浮动体 (Float) 原理

- `[h]`: 尽量放在当前处。
- `[H]`: 强制固定 (需 `float` 宏包), 适合对位置要求极高的逻辑推导。

排版建议

若编译时间过长, 可在 `\documentclass` 中临时开启 `draft` 选项, 跳过图片渲染。

§6.1

交叉引用：Label 与 Ref 的自动编号

L^AT_EX 的自动化核心在于“标签-引用”机制。通过给图表、公式或章节定义一个唯一的标签 (`\label`)，你可以在正文任何地方调用它的编号。无论你后期如何增删内容，L^AT_EX 都会在编译时自动重新计算所有编号，确保引用永不失效。

1 Greek and Hebrew letters

α	<code>\alpha</code>	ϵ	<code>\epsilon</code>	φ	<code>\varphi</code>	f	<code>\digamma</code>	Δ	<code>\Delta</code>	Θ	<code>\Theta</code>
β	<code>\beta</code>	λ	<code>\lambda</code>	ρ	<code>\rho</code>	ε	<code>\varepsilon</code>	Γ	<code>\Gamma</code>	Υ	<code>\Upsilon</code>
χ	<code>\chi</code>	μ	<code>\mu</code>	σ	<code>\sigma</code>	κ	<code>\kappa</code>	Λ	<code>\Lambda</code>	Ξ	<code>\Xi</code>
δ	<code>\delta</code>	ν	<code>\nu</code>	τ	<code>\tau</code>	φ	<code>\varphi</code>	Ω	<code>\Omega</code>	$\aleph$	<code>\aleph</code>
e	<code>\epsilon</code>	o	<code>\o</code>	θ	<code>\theta</code>	ϖ	<code>\varpi</code>	Φ	<code>\Phi</code>	$\aleph$	<code>\aleph</code>
η	<code>\eta</code>	ω	<code>\omega</code>	υ	<code>\upsilon</code>	g	<code>\g</code>	Π	<code>\Pi</code>	$\beth$	<code>\beth</code>
γ	<code>\gamma</code>	ϕ	<code>\phi</code>	ξ	<code>\xi</code>	ς	<code>\varsigma</code>	Ψ	<code>\Psi</code>	$\aleph$	<code>\aleph</code>
ι	<code>\iota</code>	π	<code>\pi</code>	ζ	<code>\zeta</code>	ϑ	<code>\vartheta</code>	Σ	<code>\Sigma</code>	$\aleph$	<code>\aleph</code>

图 6.1: 系统架构示意图

如或图 6.1所示，该架构展示了系统的核心组成部分。

$$E = mc^2 \quad (6.1)$$

质能方程（见公式 6.1）揭示了质量与能量的等价性。

§6.2

参考文献：BibTeX 数据库与引用样式

管理参考文献最专业的方式是使用 BibTeX。1. 创建数据库:新建一个 .bib 文件(如 refs.bib)，存入文献条目。2. 正文引用:使用 `\cite{key}` 命令。3. 生成列表:在文档末尾指定样式并导入数据库。

Listing 6.1: BibTeX 数据库条目示例

```
@article{Einstein1905,  
  author = "Albert Einstein",  
  title = "Does the Inertia of a Body Depend upon Its Energy-  
    Content?",
```



```
journal = "Annalen der Physik",  
year    = "1905"  
}
```

引用演示：根据爱因斯坦的研究 [?], 我们可知能量的变化会导致质量的变化。

§6.3 自动生成目录与索引

只要正确使用了 `\chapter` 和 `\section` 等命令, $\text{\LaTeX}$ 就能自动提取标题并生成带页码的目录。

- `\tableofcontents`: 生成章节目录。
- `\listoffigures`: 生成图片清单。
- `\listoftables`: 生成表格清单。

使用技巧

由于 $\text{\LaTeX}$ 需要先扫描标签再填入编号, 涉及交叉引用和目录时, 通常需要编译两次才能完全显示正确结果。

这个部分你要用的话可以单独学? maybe 我到时候教你? hhh